Data Structures and Algorithms Using C

UNIT-I

Prepared By-Ms. Anjli

Assistant Professor

What is a Data Structure?

- Data Structure is defined as a particular way of storing and organizing data in our devices to use the data efficiently and effectively.
- The main idea behind using data structures is to minimize the time and space complexities.
- An efficient data structure takes minimum memory space and requires minimum time to execute the data.

Complexities:-

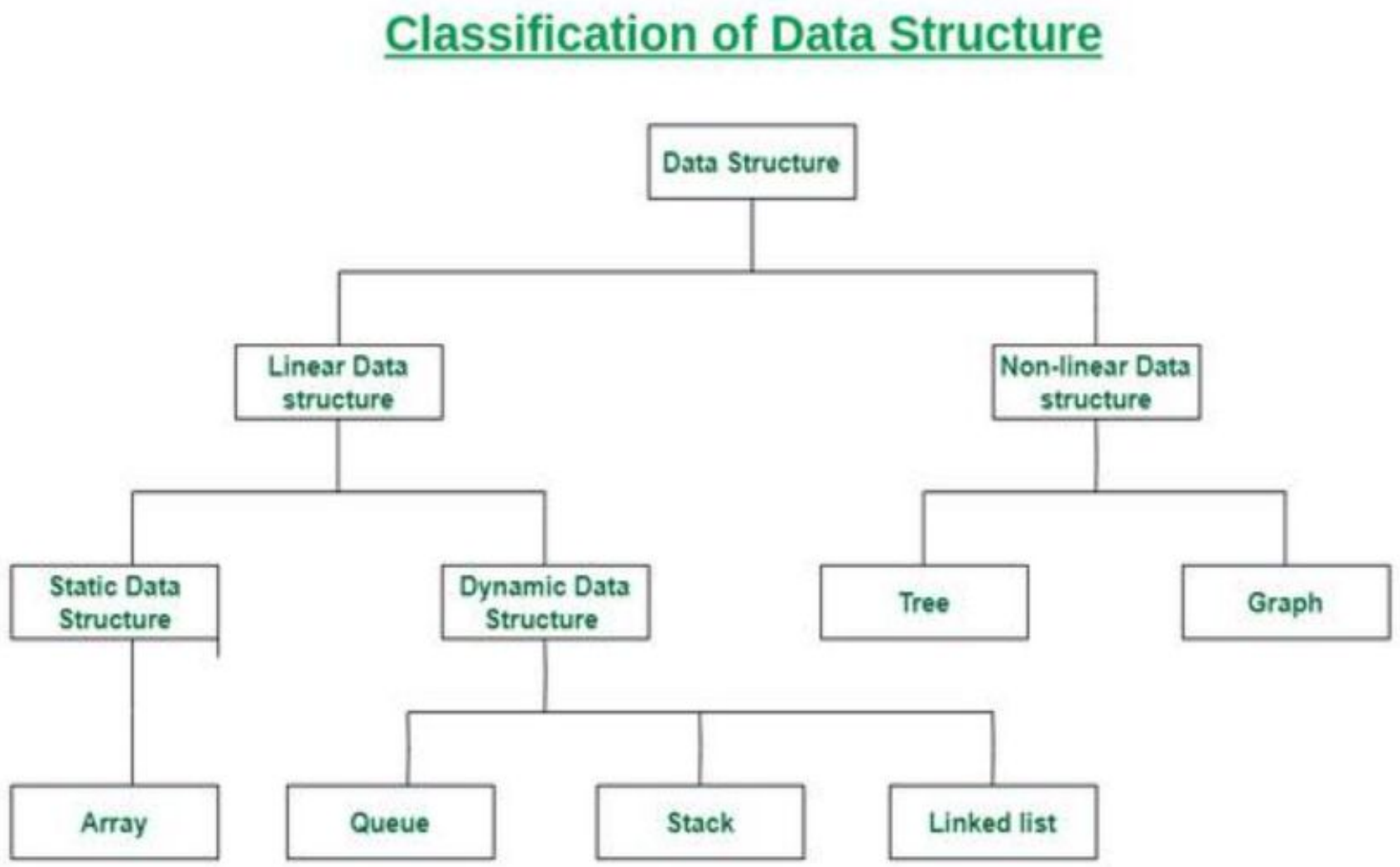
- **Time Complexity-**

  The time required by the algorithm to solve given problem is called ***time complexity*** of the algorithm.

- **Space Complexity-**

  The amount of memory required by the algorithm to solve given problem is called **space complexity** of the algorithm.

Classification of Data Structure



Linear Data Structure

- Data structure where data elements are arranged sequentially or linearly where each and every element is attached to its previous and next adjacent is called a **linear data structure**.
- In linear data structure, single level is involved.
- Therefore, we can traverse all the elements in single run only.
- Linear data structures are easy to implement because computer memory is arranged in a linear way.
- Its examples are array, stack, queue, linked list, etc.

Non-Linear Data Structure

- Data structures where data elements are not arranged sequentially or linearly are called **non-linear data structures**.
- In a non-linear data structure, single level is not involved.
- Therefore, we can’t traverse all the elements in single run only.
- Non-linear data structures are not easy to implement in comparison to linear data structure.
- It utilizes computer memory efficiently in comparison to a linear data structure.
- Its examples are trees and graphs.

Operations on Data Structure:-

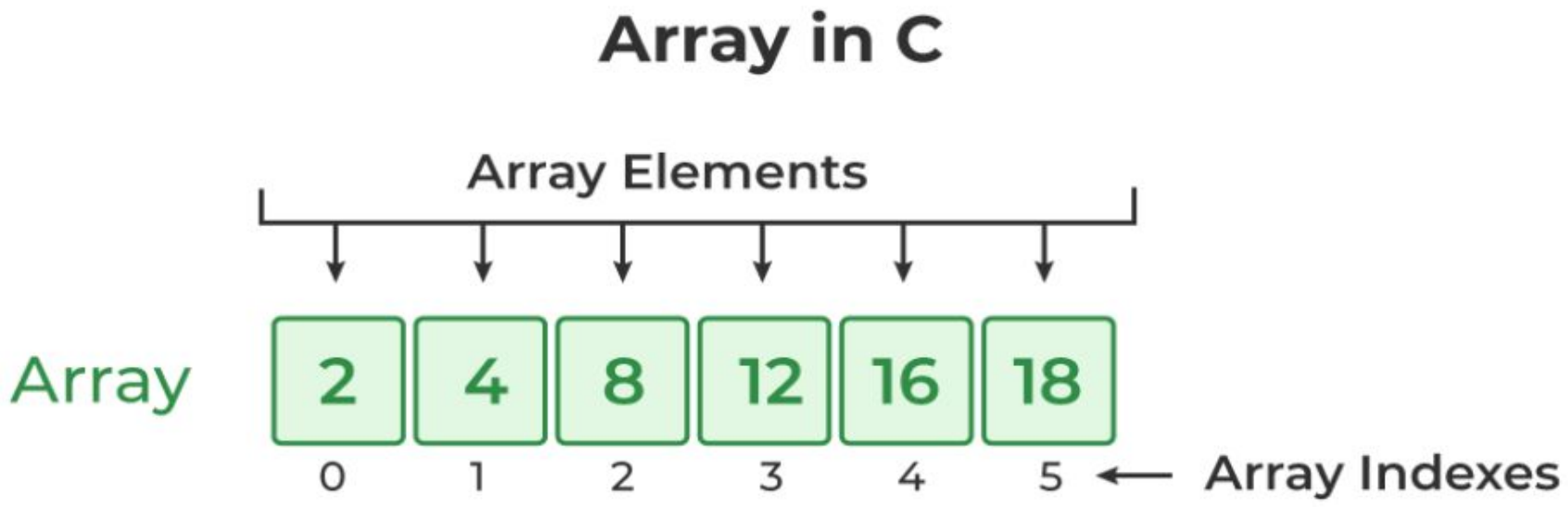
- Traversing- Visiting each and every element atleast and atmost once.
- Insertion- Adding an element in a data structure. (Overflow)
- Deletion- Removing an element from a data structure. (Underflow)
- Searching- Finding an element in a data structure. (Successful and Unsuccessful)
- Merging- Combining two data structures of same type into one.
- Sorting- Arranging data structure in either ascending or descending order.

- Overflow Condition- When data structure is full, then further insertions are not possible.
- Underflow Condition- When data structure is empty, and no further deletions are possible.

Array

- An array is a linear data structure that is defined as a collection of items of the same variable type that are stored at contiguous memory locations.
- It is one of the simplest data structures where each data element can be randomly accessed by using its index number.
- It can be used to store the collection of primitive data types such as int, char, float, etc., and also derived and user-defined data types such as pointers, structures, etc.

- Basic Terminologies-
- Array Index- It starts from 0.
- Array Element- Items stored in an array and can be accessed by their index.
- Array Length- Number of elements it can contain.



Types of Array

- **Single Dimension/ One Dimension**

The elements are arranged in a single row or column and can be accessed using a single index.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int i,numbers[5]={10,20,30,40,50};
    for(i=0;i<5;i++)
    {
        printf("Numbers[%d]=%d \n",i,numbers[i]);

    }
}
```

```
Numbers[0]=10
Numbers[1]=20
Numbers[2]=30
Numbers[3]=40
Numbers[4]=50



...Program finished with exit code 0
Press ENTER to exit console.
```

Accessing elements of 1-D array

```
#include<stdio.h>
void main()
{
    int numbers[5]={10,20,30,40,50};
    {
        printf("The third element of the array is %d \n",numbers[2]);

    }
}
```

```
The third element of the array is 30


...Program finished with exit code 0
Press ENTER to exit console.
```

Modifying Elements of 1D array

```
#include<stdio.h>
void main()
{
    int numbers[5]={10,20,30,40,50};
    {
        printf("The third element of the array is %d \n",numbers[2]);
        numbers[2]=35;
        printf("The third element of the array is now %d \n", numbers[2]);

    }
}
```

```
The third element of the array is 30
The third element of the array is now 35



...Program finished with exit code 0
Press ENTER to exit console.
```

Two-Dimensional (2D) Array

- A **two-dimensional array** or **2D array** is the simplest form of the multidimensional array.
- A **2D array** (two-dimensional array) in programming is essentially a collection of elements organized in rows and columns, forming a table-like structure.
- SYNTAX-

```
data_type array_name[rows][columns];
```

- Example-

```
int array[3][3];
```

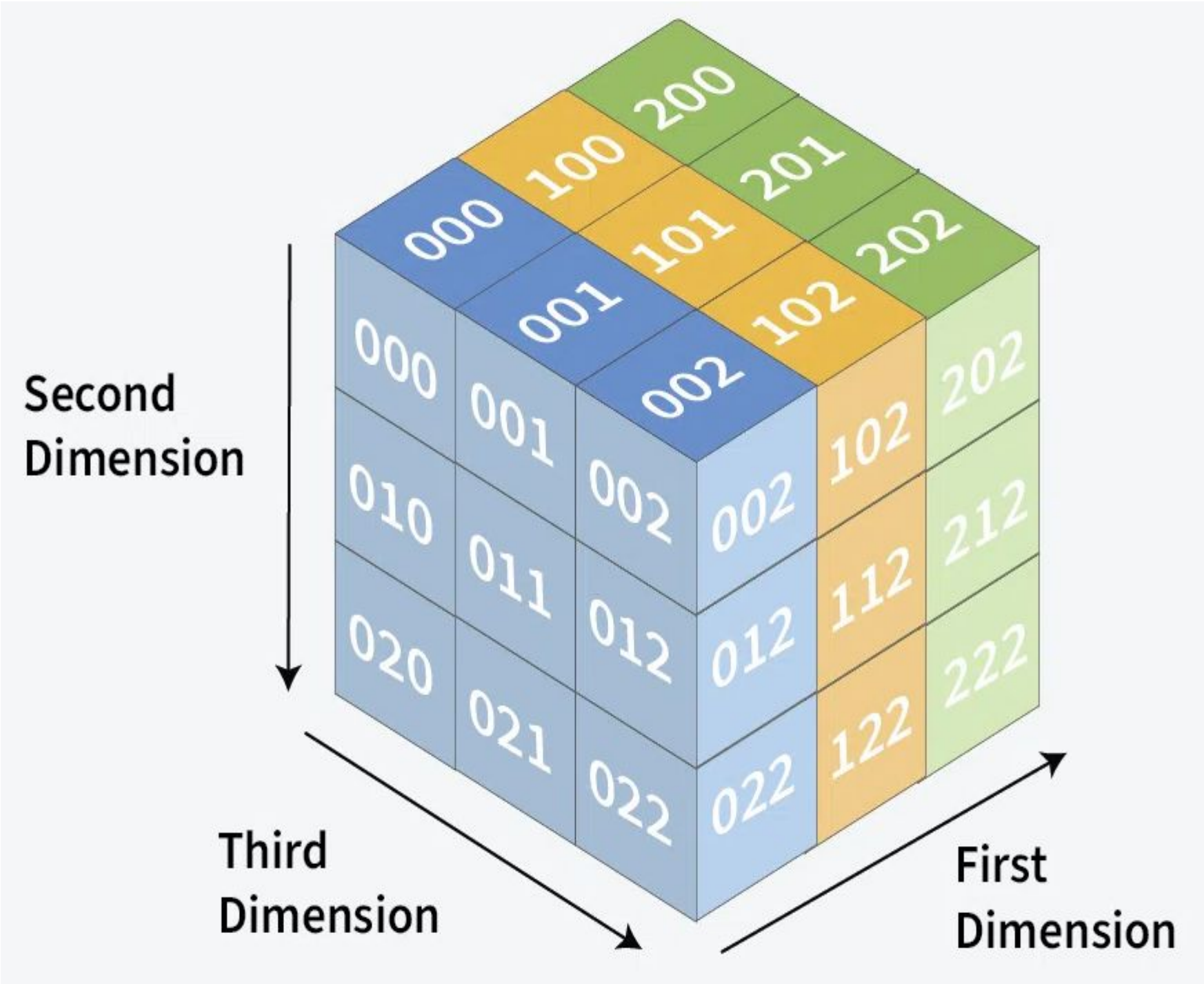
EXAMPLE-

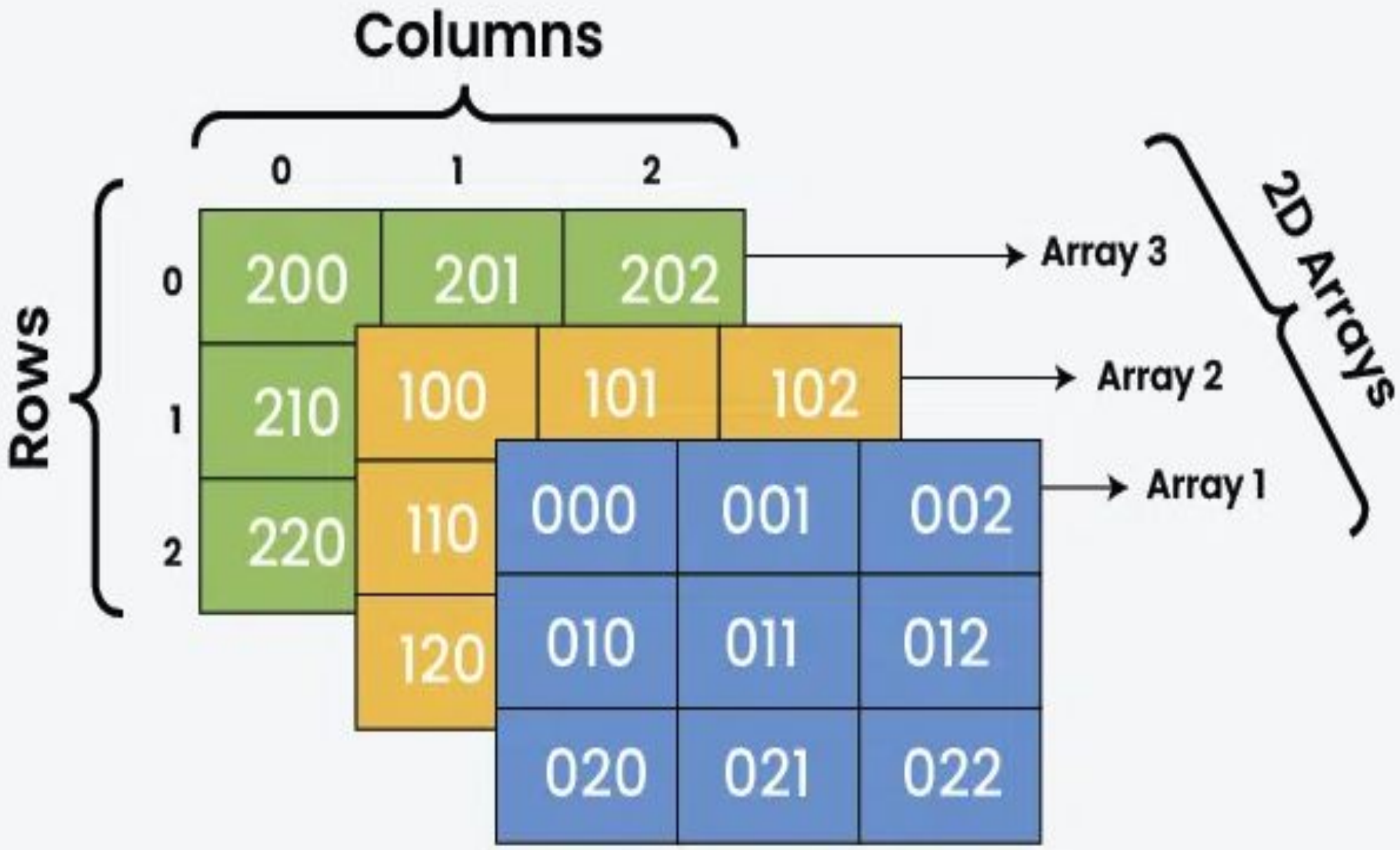
```c
#include <stdio.h>

int main() {
    // Declare and initialize a 2D array
    int matrix[2][3] = {
        {1, 2, 3},
        {4, 5, 6}
    };

    // Access elements
    printf("Element at [0][2]: %d\n", matrix[0][2]); // Access 3rd element of the 1st row
    printf("Element at [1][1]: %d\n", matrix[1][1]); // Access 2nd element of the 2nd row

    return 0;
}
```

```
Element at [0][2]: 3
Element at [1][1]: 5



...Program finished with exit code 0
Press ENTER to exit console.
```

Multidimension Array

- A **multidimensional array** in C is an array containing arrays, allowing the representation of data in multiple dimensions, such as 2D, 3D, or higher.
- **3D Arrays (Three-Dimensional Arrays):**
- Represented as a cube-like structure with layers (pages).
- data_type array_name[depth][rows][columns];
- int array[2][3][4]; // 2 layers, each with a 3x4 grid


Second
Dimension
Third
Dimension
First
Dimension
000
100
200
000
101
201
001
102
202
002
000
001
002
010
011
012
020
021
022
002
012
022
102
112
122
202
212
222


Columns
0
1
2
Rows
0
1
2
200
201
202
210
100
101
102
220
110
000
001
002
120
010
011
012
020
021
022
Array 3
Array 2
Array 1
2D Arrays

Sparse Matrix

- A sparse matrix is a matrix in which most of the elements are zero.
- Sparse matrices are common in various applications, such as scientific computing, graph algorithms, and text processing where data tends to be mostly zero.
- While performing operations on sparse matrices such as addition, subtraction, and multiplication stored as a 2D array, many operations are performed on elements with zero values which results in the amount of wastage in time.
- Better strategy is to explicitly store the non-zero elements, which greatly reduces the required storage space and computations needed to perform various operations.

Types of Sparse Matrices:

- Lower triangular matrices-
  One where all elements above the main diagonal are zero value.

$$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 9 & -2 & 0 & 0 & 0 \\ -2 & 1 & 3 & 0 & 0 \\ 3 & 1 & -1 & 6 & 0 \\ 0 & 6 & 7 & 2 & 7 \end{bmatrix}$$

Lower Triangular Matrix

- Upper triangular sparse matrices-

One where all the elements below the main diagonal are zero value.

$$\begin{bmatrix} 2 & -1 & -5 & 9 & 7 \\ 0 & 4 & 2 & 4 & 6 \\ 0 & 0 & 3 & 1 & 5 \\ 0 & 0 & 0 & 6 & 4 \\ 0 & 0 & 0 & 0 & -2 \end{bmatrix}$$

Upper Triangular Matrix

- Tri-diagonal sparse matrices-

One where all non-zero elements lie on one of the diagonals, the main diagonal above and below.

$$\begin{bmatrix} 2 & 4 & 0 & 0 & 0 \\ 3 & -1 & 5 & 0 & 0 \\ 0 & 6 & 2 & 1 & 0 \\ 0 & 0 & 1 & -4 & 5 \\ 0 & 0 & 0 & 6 & 7 \end{bmatrix}$$

Tridiagonal Matrix

- Irregular sparce matrices-

The irregular sparse matrices are the ones that behave an irregular or unstructured pattern of occurrences of non-zero elements.

$$\begin{bmatrix} 5 & 0 & 0 & 24 & 0 & 7 \\ 2 & -2 & 0 & 4 & 0 & 33 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 1 & 22 & 0 \end{bmatrix}$$

Array-Insertion

```c
#include <stdio.h>
#include<conio.h>
void insert(int arr[], int *n, int pos, int val) {

    // Shift elements to the right
    for (int i = *n; i > pos; i--)
        arr[i] = arr[i - 1];
    // Insert val at the specified position
    arr[pos] = val;
    // Increase the current size
    (*n)++;
}

int main() {
    int arr[7] = {10, 20, 30, 40, 50};
    int n = 5;
    int pos = 3;
    int val = 25;

    // Insert the value at the specified position
    insert(arr, &n, pos, val);

    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

Output-

```
10 20 30 25 40 50

...Program finished with exit code 0
Press ENTER to exit console.
```

Arrays-Deletion

```c
#include <stdio.h>
#include<conio.h>
int main() {
    int arr[] = { 10, 20, 30, 40 ,50};
    int n = sizeof(arr)/sizeof(arr[0]);
    int pos = 2;

    printf("Array before deletion\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    // Delete the element at the given position
    for (int i = pos; i < n; i++) {
        arr[i - 1] = arr[i];
    }

    if (pos <= n)
        n--;

    printf("\nArray after deletion\n");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    return 0;
}
```

Output-

```
Array before deletion
10 20 30 40 50
Array after deletion
10 30 40 50

...Program finished with exit code 0
Press ENTER to exit console.
```

Arrays- Traversing

```
#include <stdio.h>
#include<conio.h>
int main() {
    int i,arr[] = {1, 2, 3, 4, 5};
    int N = sizeof(arr) / sizeof(arr[0]);

    // Traverse and print elements using a for loop
    printf("Array elements: ");
    for (i = 0; i < N; i++)
    {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

Output-

```
Array elements: 1 2 3 4 5


...Program finished with exit code 0
Press ENTER to exit console.
```

Linear Search

- In Linear Search, we iterate over all the elements of the array and check if it the current element is equal to the target element.
- If we find any element to be equal to the target element, then return the index of the current element.
- Otherwise, if no element is equal to the target element, then return -1 as the element is not found.
- Linear search is also known as **sequential search**.

01 Step | Compare the key with each element one by one starting from the 1st element.

Key

30 Not Equal

0	1	2	3	4	5	6	7	8
10	50	30	70	80	60	20	90	40

Cur

02 Step | Compare the key with 2nd element which is not equal to the key, so move to the next element

Key

30 Not Equal

0	1	2	3	4	5	6	7	8
10	50	30	70	80	60	20	90	40

Cur

03 Step | Compare the key with the 3rd element. Key is found so stop the search.

Key

30 Equal

0	1	2	3	4	5	6	7	8
10	50	30	70	80	60	20	90	40

Cur

```c
#include <stdio.h>
#include<conio.h>
int linearSearch(int* arr, int n, int key) {
    for (int i = 0; i < n; i++) {
        // If key is found, return key
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}

int main() {
    int arr[] = { 10, 50, 30, 70, 80, 60, 20, 90, 40 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int key = 30;

    int i = linearSearch(arr, n, key);

    if (i == -1)
        printf("Key Not Found");
    else
        printf("Key Found at Index: %d", i);

    return 0;
}
```

OUTPUT-

```
Key Found at Index: 2

...Program finished with exit code 0
Press ENTER to exit console.
```

Binary Search

- **Binary search** is a search algorithm used to find the position of a target value within a **sorted** array.
- It works by repeatedly dividing the search interval in half until the target value is found or the interval is empty.
- The search interval is halved by comparing the target element with the middle value of the search space.
- To apply Binary Search algorithm, The data structure must be sorted.

Initially | Find Key = 23 using Binary Search

	0	1	2	3	4	5	6	7	8	9
arr[] =	2	5	8	12	16	23	38	56	72	91

Step 1 | Key (i.e., 23) is greater than current mid element (i.e., 16). The search space moves to the right.

Key

23

	0	1	2	3	4	5	6	7	8	9
arr[] =	2	5	8	12	16	23	38	56	72	91

Low = 0 Mid = 4 High = 9

Step 2 Key is less than the current mid 56. The search space moves to the left.

Step 3 If the key matches the value of the mid element, the element is found and stop search.

```c
#include <stdio.h>
#include<conio.h>
int binarySearch(int arr[], int left, int right, int key) {

    // Loop will run till left > right. It means that there
      // are no elements to consider in the given subarray
    while (left <= right) {

        // calculating mid point
        int mid = left + (right - left) / 2;

        // Check if key is present at mid
        if (arr[mid] == key) {
            return mid;
        }

        // If key greater than arr[mid], ignore left half
        if (arr[mid] < key) {
            left = mid + 1;
        }

        // If key is smaller than or equal to arr[mid],
        // ignore right half
        else {
            right = mid - 1;
        }
    }
```

```c
    return -1;
}


int main() {
    int arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91};
    int size = sizeof(arr) / sizeof(arr[0]);

    // Element to be searched
    int key = 23;

    int result = binarySearch(arr, 0, size - 1, key);
    if (result == -1) {
        printf("Element is not present in array");
    }
    else {
        printf("Element is present at index %d", result);
    }
    return 0;
}
```

OUTPUT-

```
Element is present at index 5

...Program finished with exit code 0
Press ENTER to exit console.
```

Arrays-Reversing

```c
#include <stdio.h>
#include<conio.h>
void rev(int arr[], int n) {

    int temp,l = 0, r = n - 1;
    while (l < r) {
        temp = arr[l];
        arr[l] = arr[r];
        arr[r] = temp;

        // Move pointers towards middle
        l++;
        r--;
    }
}
int main() {
    int i,n,arr[] = {1, 2, 3, 4, 5};
     n = sizeof(arr) / sizeof(arr[0]);

      // Reverse array arr
    rev(arr, n);

    for (i = 0; i < n; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

OUTPUT-

```
5 4 3 2 1

...Program finished with exit code 0
Press ENTER to exit console.
```

Dynamic Memory Allocation

- Dynamic Memory Allocation (DMA) allows a program to allocate memory at **runtime** instead of compile-time. This is useful when we don't know the exact amount of memory needed in advance (e.g., working with **linked lists, trees, graphs, and dynamic arrays**).
- 📌 **Key Benefits:**
  ✔️ Memory is allocated when required and freed when no longer needed.
  ✔️ More efficient memory management.

- There are 4 library functions provided by C defined under **<stdlib.h>** header file to facilitate dynamic memory allocation in C programming. They are:

1. malloc()
2. calloc()
3. free()
4. realloc()

malloc()- Memory Allocation

- Allocates a block of memory but **does not initialize it**. The memory contains garbage values.
- Syntax-

  ptr = (cast_type*) malloc(size_in_bytes);
- Example

  For Example:

ptr = (int*) malloc(100 * sizeof(int));

Since the size of int is 4 bytes, this statement will allocate 400 bytes of memory. And, the pointer ptr holds the address of the first byte in the allocated memory.

Malloc()

4 bytes

```
int* ptr = ( int* ) malloc ( 5* sizeof ( int ));
```

ptr =

A large 20 bytes memory block is dynamically allocated to ptr

20 bytes of memory

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int* ptr;
    int n, i;

    printf("Enter number of elements:");
    scanf("%d",&n);
    printf("Entered number of elements: %d\n", n);

    // Dynamically allocate memory using malloc()
    ptr = (int*)malloc(n * sizeof(int));

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        // Memory has been successfully allocated
        printf("Memory successfully allocated using malloc.\n");
```

```c
    // Get the elements of the array
    for (i = 0; i < n; ++i) {
        ptr[i] = i + 1;
    }

    // Print the elements of the array
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }
}

return 0;
}
```

```
Enter number of elements:4
Entered number of elements: 4
Memory successfully allocated using malloc.
The elements of the array are: 1, 2, 3, 4,

...Program finished with exit code 0
Press ENTER to exit console.
```

Calloc()- Contiguous allocation

“calloc” or **“contiguous allocation”** method in C is used to dynamically allocate the specified number of blocks of memory of the specified type. it is very much similar to malloc() but It initializes each block with a default value ‘0’.

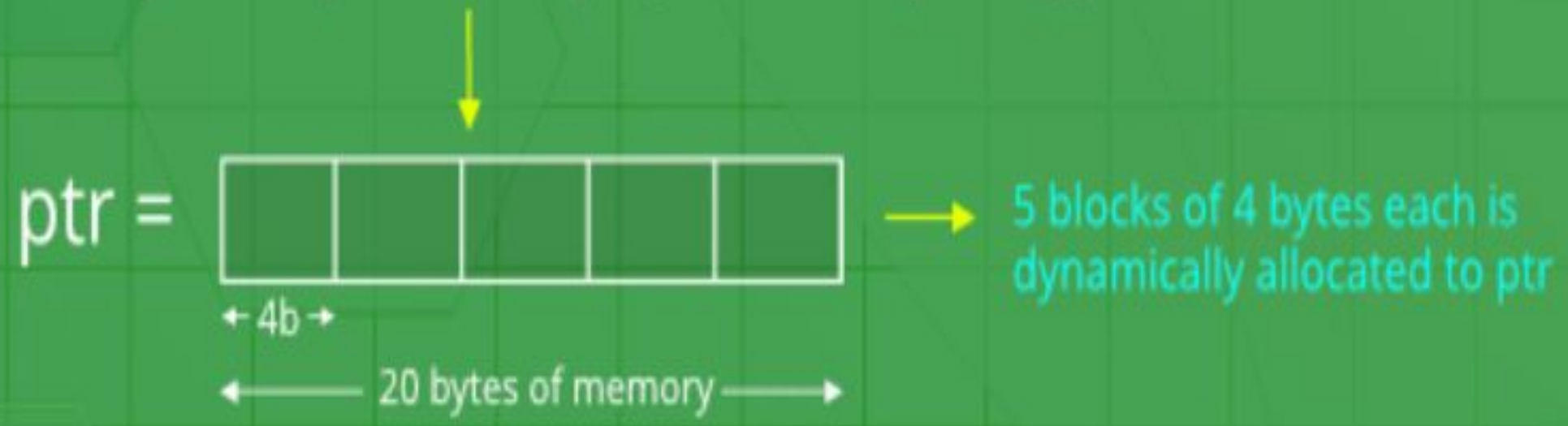
- Syntax-
  ```
  ptr = (cast-type*)calloc(n, element-size);
  ```

- ***Example***
  ptr = (float*) calloc(25, sizeof(float));
  This statement allocates contiguous space in memory for 25 elements each with the size of the float.

Calloc()

4 bytes

```
int* ptr = ( int* ) calloc ( 5, sizeof ( int ));
```



```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int* ptr;
    int n, i;

    // Get the number of elements for the array
    n = 5;
    printf("Number of elements: %d\n", n);

    ptr = (int*)calloc(n, sizeof(int));

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        printf("Memory successfully allocated using calloc.\n");

        // Get the elements of the array
        for (i = 0; i < n; ++i) {
            ptr[i] = i + 1;
        }
```

```
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }
}

return 0;
}
```

```
Number of elements: 5
Memory successfully allocated using calloc.
The elements of the array are: 1, 2, 3, 4, 5,

...Program finished with exit code 0
Press ENTER to exit console.
```

free()

- **“free”** method in C is used to dynamically **de-allocate** the memory.
- The memory allocated using functions malloc() and calloc() is not de-allocated on their own.
- Hence the free() method is used, whenever the dynamic memory allocation takes place.
- It helps to reduce wastage of memory by freeing it.
- Syntax-

```
free(ptr);
```

Free()

4 bytes

```
int* ptr = ( int* ) calloc ( 5, sizeof ( int ));
```

ptr =

5 blocks of 4 bytes each is dynamically allocated to ptr

operation on ptr

free(ptr)

The memory of ptr is released

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int *ptr, *ptr1;
    int n, i;

    n = 5;
    printf("Enter number of elements: %d\n", n);

    ptr = (int*)malloc(n * sizeof(int));

    ptr1 = (int*)calloc(n, sizeof(int));

    if (ptr == NULL || ptr1 == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
```

```
else {

    printf("Memory successfully allocated using malloc.\n");

    free(ptr);
    printf("Malloc Memory successfully freed.\n");

    printf("\nMemory successfully allocated using calloc.\n");

    free(ptr1);
    printf("Calloc Memory successfully freed.\n");
}

return 0;
}
```

```
Enter number of elements: 5
Memory successfully allocated using malloc.
Malloc Memory successfully freed.

Memory successfully allocated using calloc.
Calloc Memory successfully freed.


...Program finished with exit code 0
Press ENTER to exit console.
```

realloc()

- **“realloc”** or **“re-allocation”** method in C is used to dynamically change the memory allocation of a previously allocated memory.
- In other words, if the memory previously allocated with the help of malloc or calloc is insufficient, realloc can be used to **dynamically re-allocate memory**.
- re-allocation of memory maintains the already present value and new blocks will be initialized with the default garbage value.

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    int* ptr;
    int n, i;

    n = 5;
    printf("Enter number of elements: %d\n", n);

    ptr = (int*)calloc(n, sizeof(int));

    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        printf("Memory successfully allocated using calloc.\n");

        for (i = 0; i < n; ++i) {
            ptr[i] = i + 1;
        }
```

```c
printf("The elements of the array are: ");
for (i = 0; i < n; ++i) {
    printf("%d, ", ptr[i]);
}

n = 10;
printf("\n\nEnter the new size of the array: %d\n", n);

// Dynamically re-allocate memory using realloc()
ptr = (int*)realloc(ptr, n * sizeof(int));

  if (ptr == NULL) {
  printf("Reallocation Failed\n");
  exit(0);
}

printf("Memory successfully re-allocated using realloc.\n");

// Get the new elements of the array
for (i = 5; i < n; ++i) {
    ptr[i] = i + 1;
}
```

```
    printf("The elements of the array are: ");
    for (i = 0; i < n; ++i) {
        printf("%d, ", ptr[i]);
    }

    free(ptr);
}

return 0;
}
```

```
Enter number of elements: 5
Memory successfully allocated using calloc.
The elements of the array are: 1, 2, 3, 4, 5,

Enter the new size of the array: 10
Memory successfully re-allocated using realloc.
The elements of the array are: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,

...Program finished with exit code 0
Press ENTER to exit console.
```

S.No	Static Memory Allocation	Dynamic Memory Allocation
1	In the static memory allocation, variables get allocated permanently, till the program executes or function call finishes.	In the Dynamic memory allocation, the memory is controlled by the programmer. It gets allocated whenever a malloc() is executed gets deallocated wherever the free() is executed.
2	Static Memory Allocation is done before program execution.	Dynamic Memory Allocation is done during program execution.
3	It uses stack for managing the static allocation of memory	It uses heap (not heap data structure) of memory for managing the dynamic allocation of memory
4	It is less efficient	It is more efficient
5	In Static Memory Allocation, there is no memory re-usability	In Dynamic Memory Allocation, there is memory re-usability and memory can be freed when not required
6	In static memory allocation, once the memory is allocated, the memory size can not change.	In dynamic memory allocation, when memory is allocated the memory size can be changed.

7	In this memory allocation scheme, we cannot reuse the unused memory.	This allows reusing the memory. The user can allocate more memory when required. Also, the user can release the memory when the user needs it.
8	In this memory allocation scheme, execution is faster than dynamic memory allocation.	In this memory allocation scheme, execution is slower than static memory allocation.
9	In this memory is allocated at compile time.	In this memory is allocated at run time.
10	In this allocated memory remains from start to end of the program.	In this allocated memory can be released at any time during the program.
11	**Example:** This static memory allocation is generally used for array.	**Example:** This dynamic memory allocation is generally used for linked list.

Sorting

- Sorting is the process of arranging elements in a specific order, typically in **ascending** or **descending** order.
- Sorting is an essential concept in **Data Structures and Algorithms (DSA)** because it improves the efficiency of searching and data retrieval operations.
- Types of Sorting Algorithms-

  Selection Sort

  Bubble Sort

  Insertion Sort

  Merge Sort

  Quick Sort

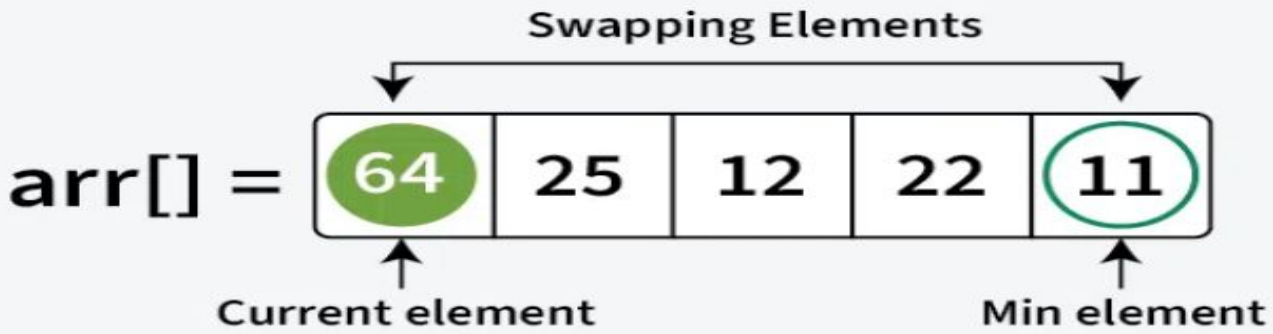
Selection Sort

- Selection Sort is a simple and efficient sorting algorithm that works by repeatedly selecting the smallest (or largest) element from the unsorted portion of the list and moving it to the sorted portion of the list.

Working

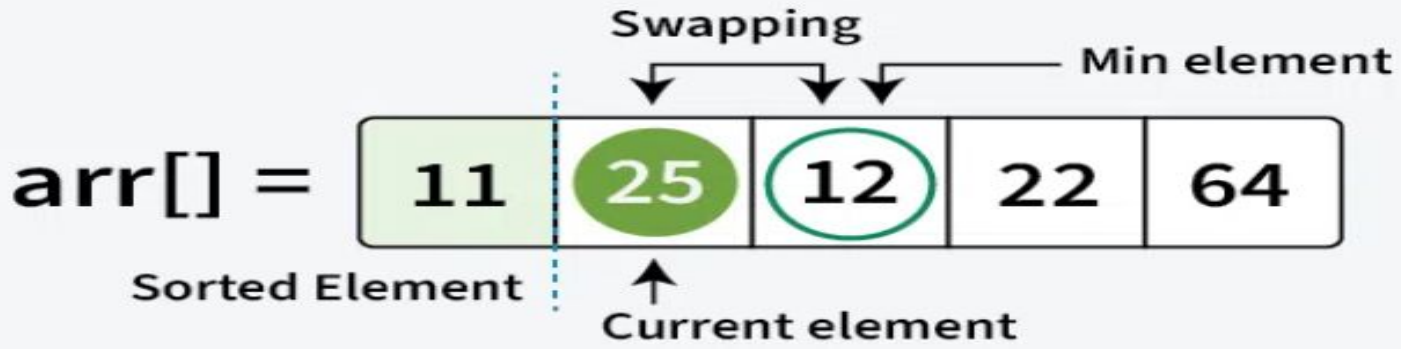
01 Step

Start from the first element at index 0, find the smallest element in the rest of the array which is unsorted, and swap (11) with current element(64).



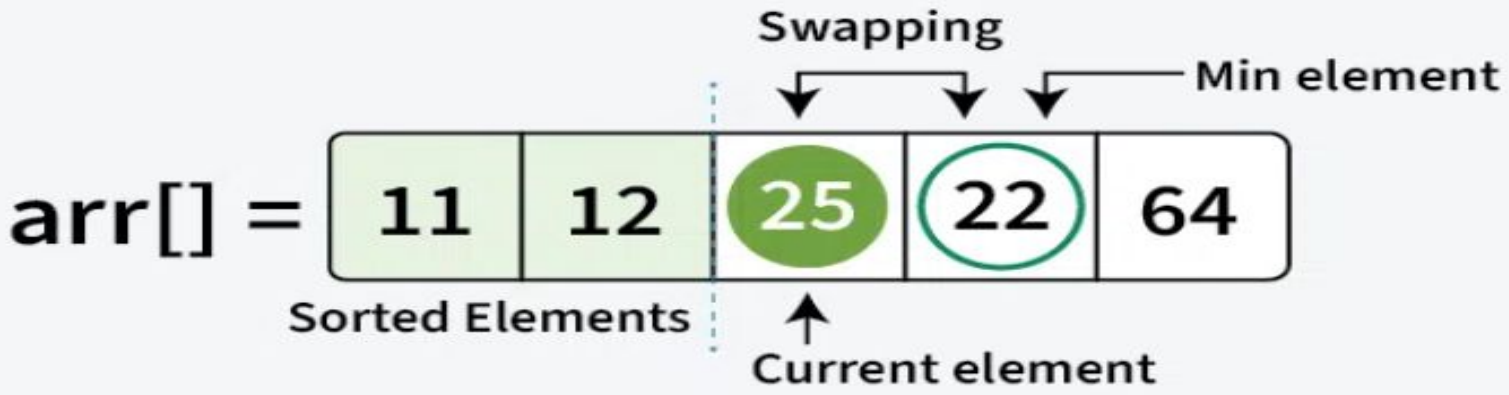
02 Step

Move to the next element at index 1 (25). Find the smallest in unsorted subarray, and swap (12) with current element (25).



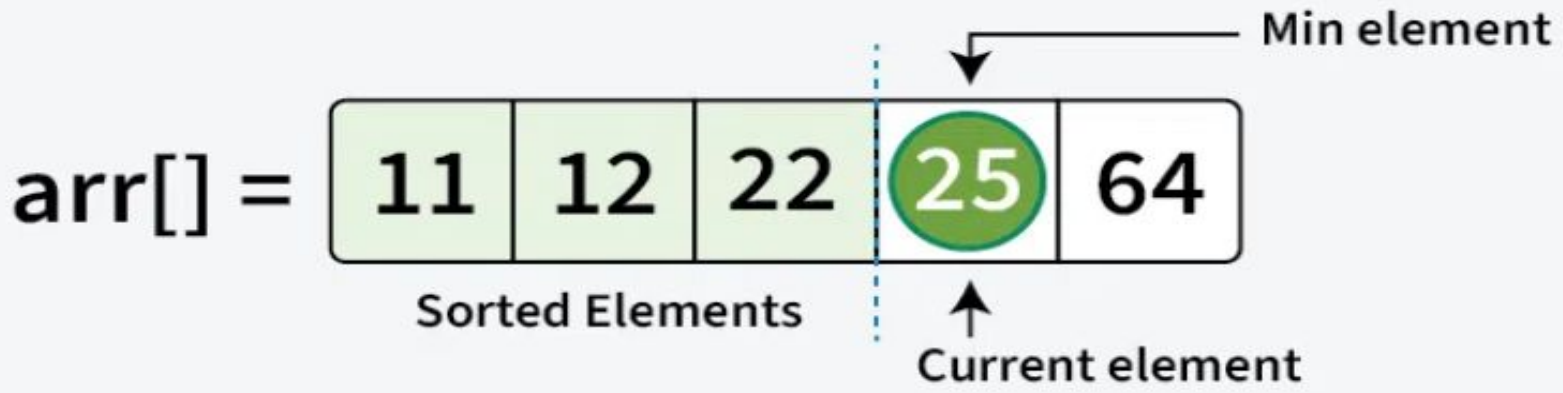
03 Step

Move to element at index 2 (25). Find the minimum element from unsorted subarray, Swap (22) with current element (25).



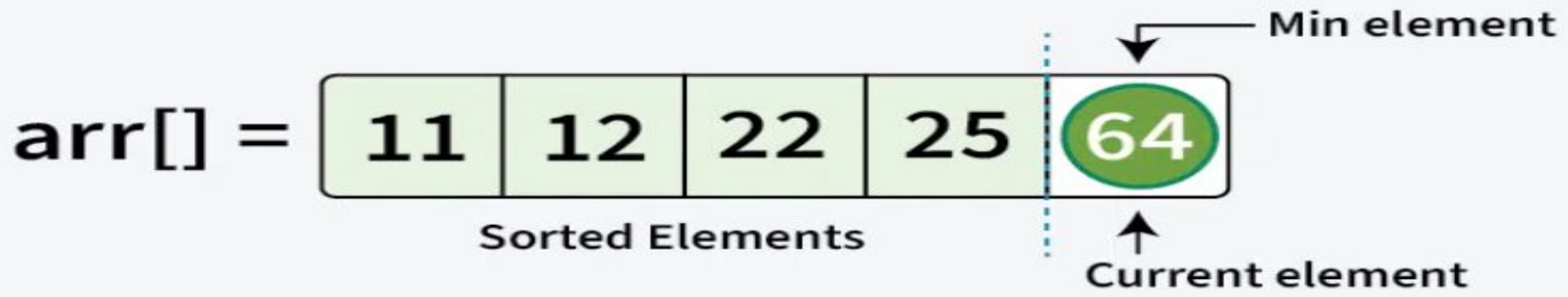
04 Step

Move to element at index 3 (25), find the minimum from unsorted subarray and swap (25) with current element (25).



05 Step

Move to element at index 4 (64), find the minimum from unsorted subarray and swap (64) with current element (64).



06 Step

We get the sorted array at the end.

Algorithm

- Start from the first element and assume it as the minimum.
- Scan the rest of the array to find the smallest element.
- Swap the smallest element with the first element.
- Move to the next element and repeat steps 2 and 3 for the remaining array.
- Repeat this process until the entire array is sorted.

```c
#include <stdio.h>
#include<conio.h>
void selectionSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {

        // Assume the current position holds
        // the minimum element
        int min_idx = i;

        // Iterate through the unsorted portion
        // to find the actual minimum
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx]) {

                // Update min_idx if a smaller element is found
                min_idx = j;
            }
        }

        // Move minimum element to its
        // correct position
         temp = arr[i];
        arr[i] = arr[min_idx];
        arr[min_idx] = temp;
    }
}
```

```c
void printArray(int arr[], int n) {
    int i;
    for ( i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

int main() {
    int arr[] = {64, 25, 12, 22, 11};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    selectionSort(arr, n);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}
```

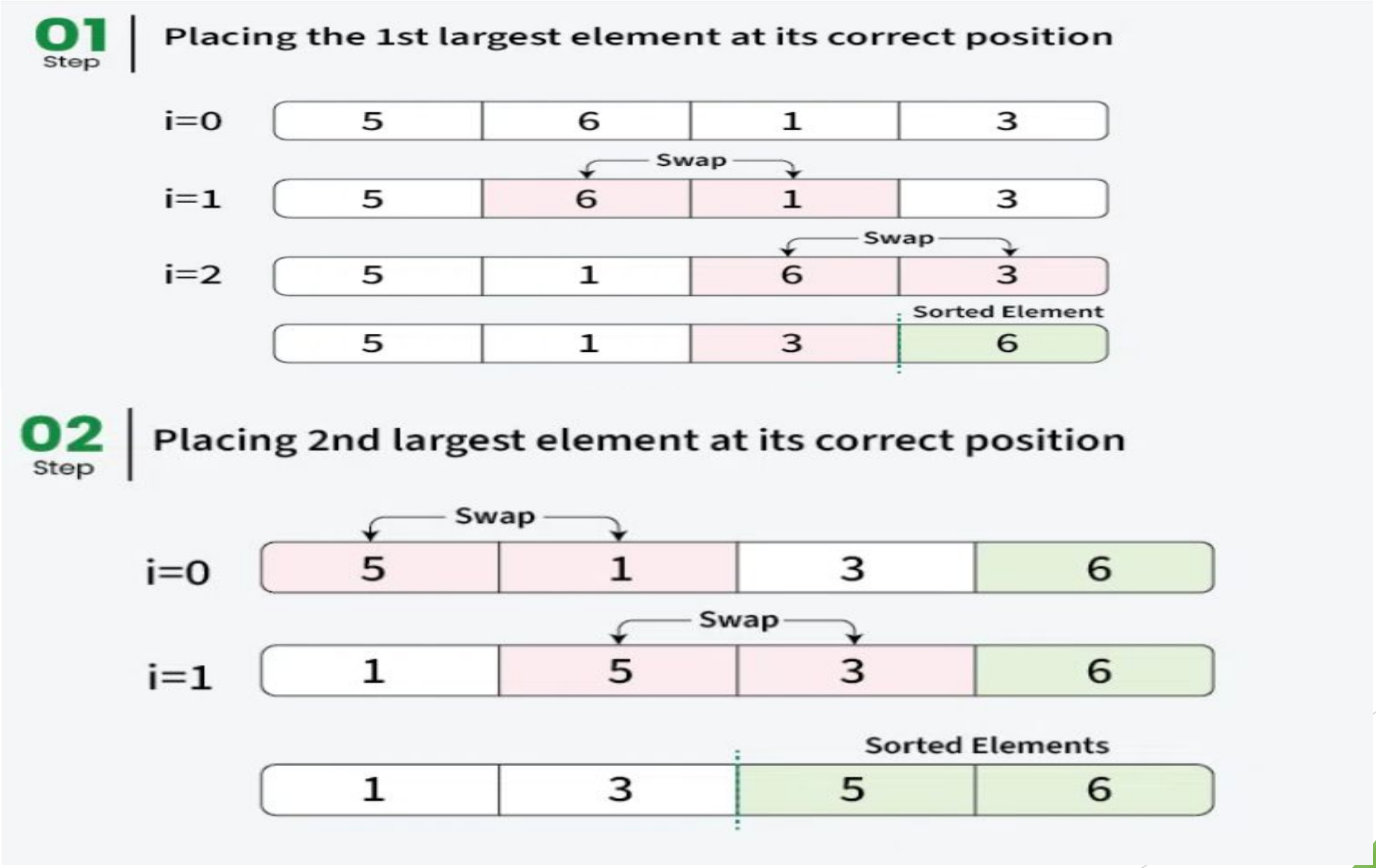
OUTPUT

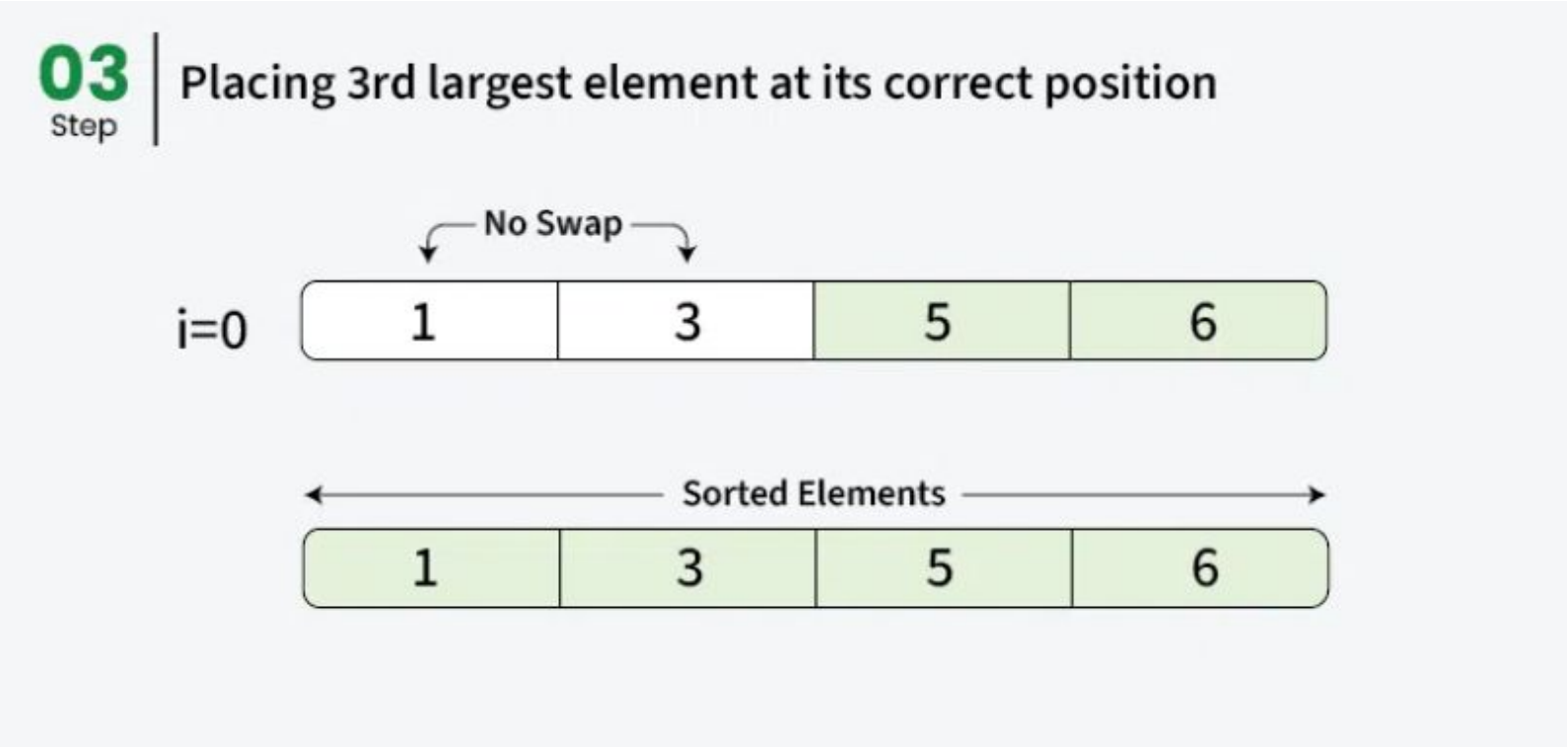
```
Original array: 64 25 12 22 11
Sorted array: 11 12 22 25 64


...Program finished with exit code 0
Press ENTER to exit console.
```

Bubble Sort

- Bubble Sort is a simple sorting algorithm that works by comparing the adjacent elements in the list and swapping them if the elements are not in the specified order.

Working


01 Step
Placing the 1st largest element at its correct position
i=0
5 6 1 3
Swap
i=1
5 6 1 3
Swap
i=2
5 1 6 3
Sorted Element
5 1 3 6
02 Step
Placing 2nd largest element at its correct position
Swap
i=0
5 1 3 6
Swap
i=1
1 5 3 6
Sorted Elements
1 3 5 6


03
Step
Placing 3rd largest element at its correct position
No Swap
i=0
1
3
5
6
Sorted Elements
1
3
5
6

Algorithm

```
begin BubbleSort(arr)
  for all array elements
    if arr[i] > arr[i+1]
      swap(arr[i], arr[i+1])
    end if
  end for
  return arr
end BubbleSort
```

```c
#include <stdio.h>
#include<conio.h>
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {  // Outer loop for passes
        for (int j = 0; j < n - i - 1; j++) {  // Inner loop for comparisons
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

// Function to print an array
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

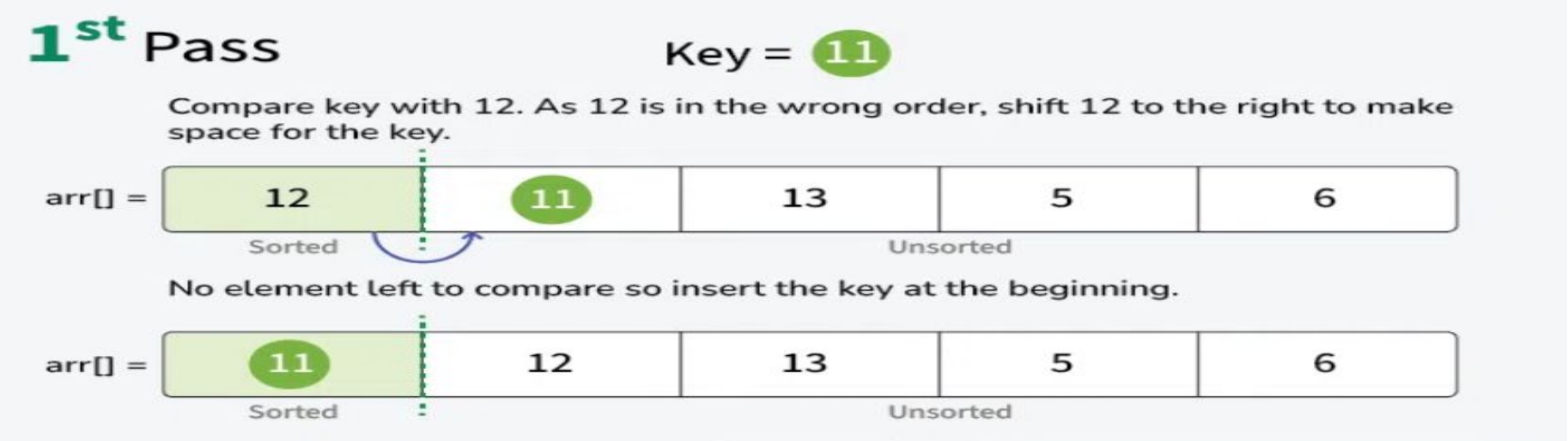
```c
int main() {
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: \n");
    printArray(arr, n);

    bubbleSort(arr, n);  // Sorting the array

    printf("Sorted array: \n");
    printArray(arr, n);

    return 0;
}
```

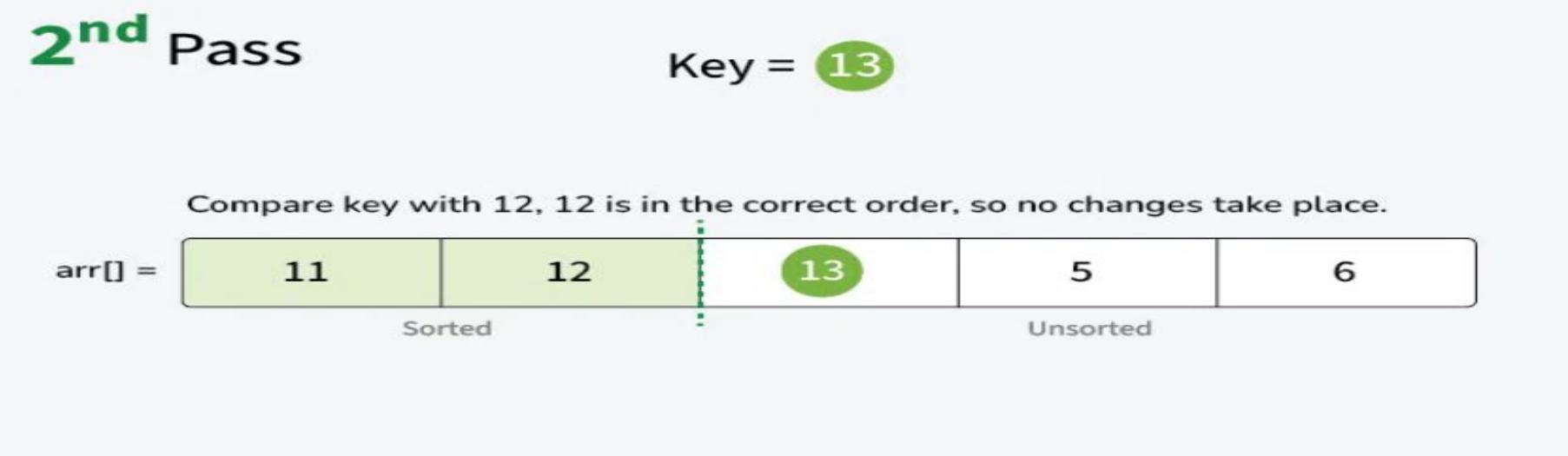
```
Original array:
64 34 25 12 22 11 90
Sorted array:
11 12 22 25 34 64 90


...Program finished with exit code 0
Press ENTER to exit console.
```

Insertion Sort

- Insertion sort is one of the simple and comparison-based sorting algorithms.
- The basic idea behind the algorithm is to virtually divide the given list into two parts: a sorted part and an unsorted part, then pick an element from the unsorted part and insert it in its place in the sorted part.
- It does this till all the elements are placed in the sorted part.

Working

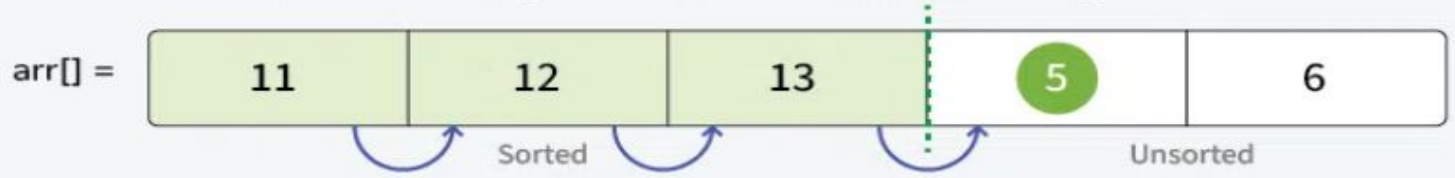

1st Pass
Key = 11
Compare key with 12. As 12 is in the wrong order, shift 12 to the right to make space for the key.
arr[] =
12
11
13
5
6
Sorted
Unsorted
No element left to compare so insert the key at the beginning.
arr[] =
11
12
13
5
6
Sorted
Unsorted


2nd Pass
Key = 13
Compare key with 12, 12 is in the correct order, so no changes take place.
arr[] =
11
12
13
5
6
Sorted
Unsorted

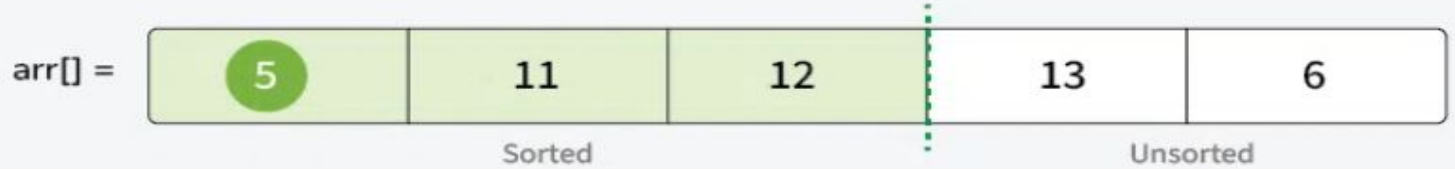
3rd Pass

Key = 5

Compare key with all the elements in the sorted subarray starting with 13, if the element is in the wrong order, shift that element to the right.



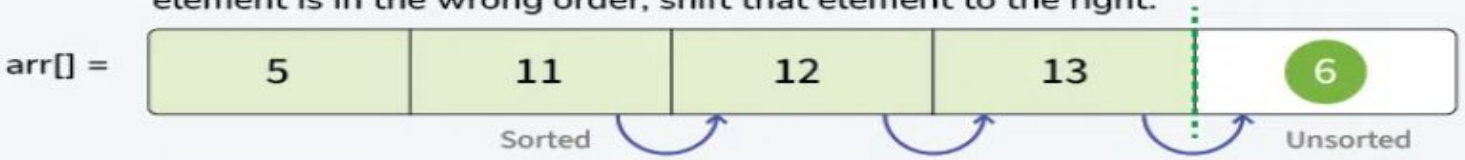
No element left to compare, so insert key at the beginning.



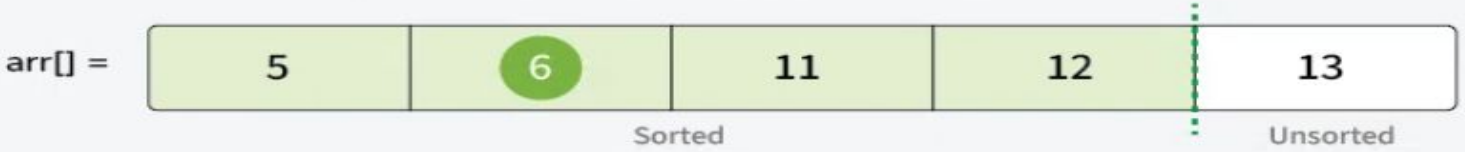
4th Pass

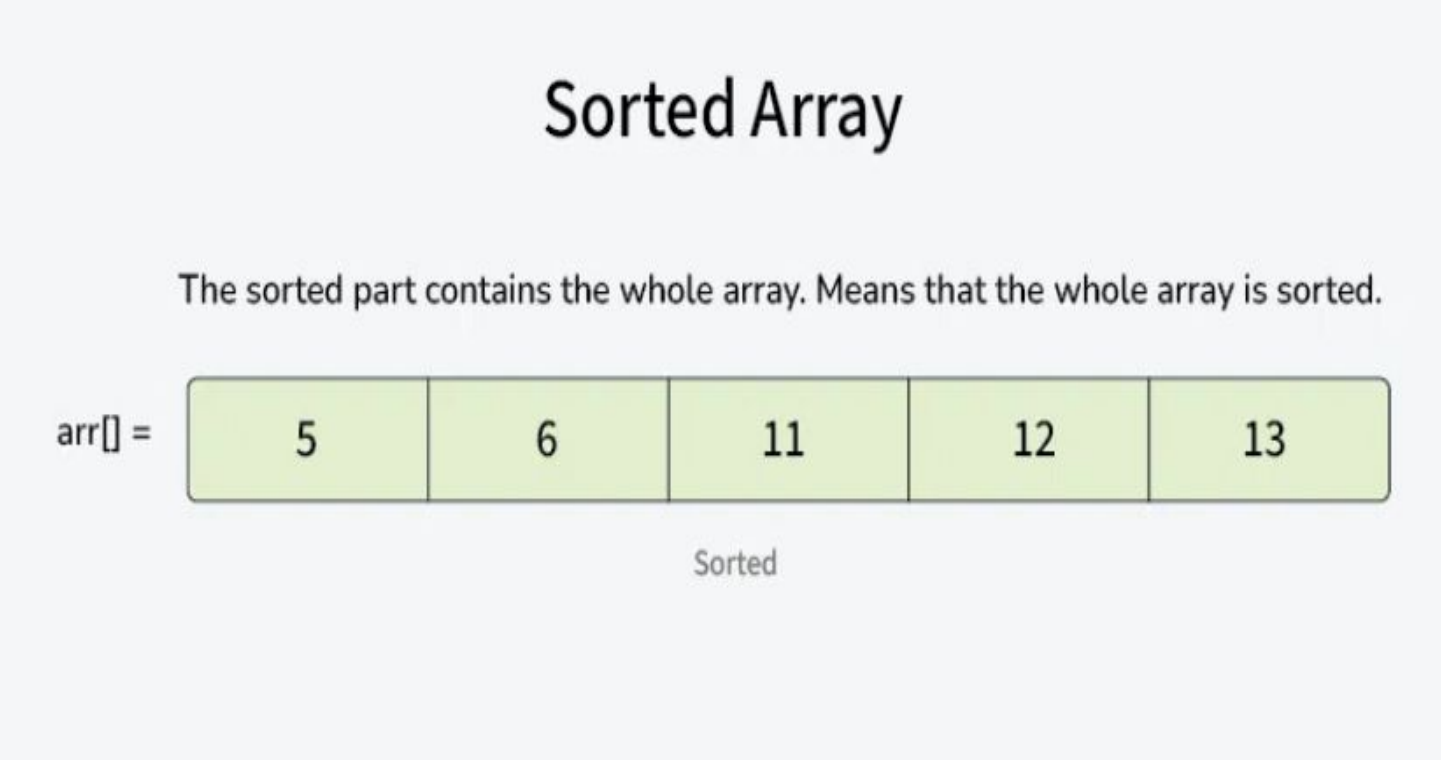
Key = 6

Compare key with all the elements in the sorted subarray starting with 13, if the element is in the wrong order, shift that element to the right.



5 is in the correct order, so shifting stops. Insert key after 5.




Sorted Array
The sorted part contains the whole array. Means that the whole array is sorted.
arr[] =
5
6
11
12
13
Sorted

Algorithm

Start with the **second element (index 1)** as the **key**.

Compare the **key** with the elements before it.

If the **key is smaller** than the compared element, shift the compared element one position to the **right**.

Insert the **key** where the shifting of elements stops.

Repeat steps 2-4 for all elements in the unsorted part of the list.

```c
#include <stdio.h>
#include<conio.h>
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;

        // Move elements that are greater than key one position ahead
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

```c
int main() {
    int arr[] = {12, 11, 13, 5, 6};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    insertionSort(arr, n);

    printf("Sorted array: ");
    printArray(arr, n);

    return 0;
}
```

```
Original array: 12 11 13 5 6
Sorted array: 5 6 11 12 13


...Program finished with exit code 0
Press ENTER to exit console.
```

Merge Sort

- **Merge Sort** is a comparison-based sorting algorithm that works by dividing the input array into two halves, then calling itself for these two halves, and finally it merges the two sorted halves.

```c
#include <stdio.h>
#include<conio.h>
// Merge function to combine two sorted halves
void merge(int arr[], int left, int mid, int right) {
    int i = left, j = mid + 1, k = 0;
    int temp[right - left + 1];

    while (i <= mid && j <= right) {
        if (arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (i = left, k = 0; i <= right; i++, k++)
        arr[i] = temp[k];
}
```

```c
void mergeSort(int arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {5, 2, 9, 1, 6, 3};
    int size = sizeof(arr) / sizeof(arr[0]);
    printf("Before sorting: ");
    printArray(arr, size);
    mergeSort(arr, 0, size - 1);
    printf("After sorting: ");
    printArray(arr, size);
    return 0;
}
```

```
Before sorting: 5 2 9 1 6 3
After sorting: 1 2 3 5 6 9



...Program finished with exit code 0
Press ENTER to exit console.
```

Quick Sort

- **QuickSort** is a sorting algorithm based on the Divide and Conquer that picks an element as a pivot and partitions the given array around the picked pivot by placing the pivot in its correct position in the sorted array. .
- There are mainly three steps in the algorithm:
- **Choose a Pivot:** Select an element from the array as the pivot. The choice of pivot can vary (e.g., first element, last element, random element, or median).
- **Partition the Array:** Re arrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right.
- **Recursively Call:** Recursively apply the same process to the two partitioned sub-arrays.
- **Base Case:** The recursion stops when there is only one element left in the sub-array, as a single element is already sorted.

Choice of Pivot

There are many different choices for picking pivots-

- Always pick the first (or last) element as a pivot. The problem with this approach is it ends up in the worst case when array is already sorted.
- Pick a random element as a pivot. This is a preferred approach because it does not have a pattern for which the worst case happens.
- Pick the median element is pivot. This is an ideal approach in terms of time complexity as we can find median in linear time and the partition function will always divide the input array into two halves. But it takes more time on average as median finding has high constants.

Time and Space Complexities

Sorting Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$